I-SUNS: Zadanie č. 1

Neurónové siete

Jozef Barčák, 128090

Úlohou je predpovedať, či návšteva e-shopu skončí nákupom. Pracujem v Pythone. Na prácu s dátami používam pandas, na jednoduchý model scikit-learn a na neurónovú sieť PyTorch.

1 Predspracovanie dát a jednoduchý model

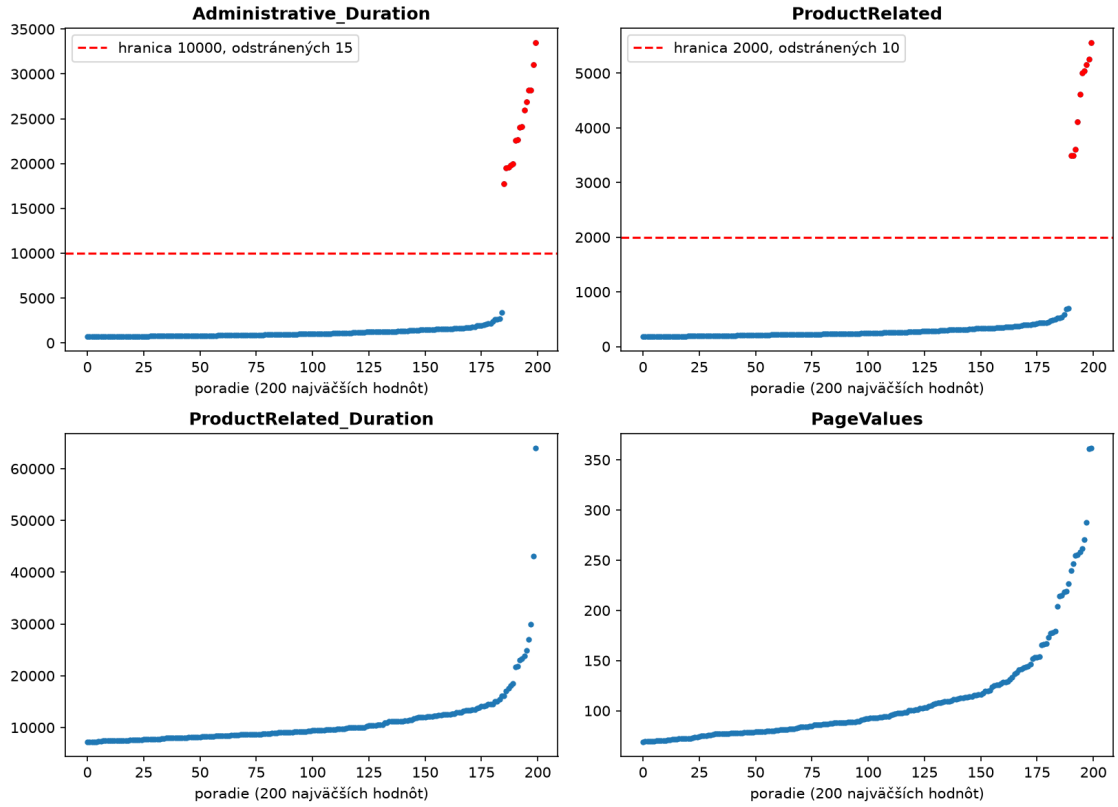
1.1 Dáta

Dataset má 12 333 riadkov a 19 stĺpcov, každý riadok je jedna návšteva. Cieľová premenná *Revenue* má hodnotu True len v 1 909 riadkoch, teda v 15,5 %. Triedy sú nevyvážené a pri hodnotení modelu s tým treba počítať.

1.2 Outliery

Najprv som podľa popisu stĺpcov skontroloval hodnoty, ktoré nemôžu existovať. Čas strávený na stránke nemôže byť záporný. *BounceRates* má byť v rozsahu od 0 do 1, v dátach boli aj hodnoty okolo −42,9. Hodnota Admin v stĺpci *VisitorType* v popise nie je a vyskytla sa raz.

Potom som hľadal príliš veľké hodnoty. Tie sa z popisu určiť nedajú, lebo počet stránok ani čas nemajú danú hornú hranicu. Hodnoty v stĺpci som si preto zoradil a hľadal medzi nimi skok (obr. 1).



Obr. 1: 200 najväčších hodnôt vo vybraných stĺpcoch, zoradených od najmenšej. Červená čiara je zvolená hranica, červené body sú odstránené hodnoty

V stĺpci *Administrative_Duration* končia bežné hodnoty na 3 399 sekundách a ďalšia je až 17 757. V stĺpci *ProductRelated* je podobná medzera medzi 705 a 3 491. Hodnoty nad medzerou som odstránil, hranice som nastavil na 10 000 a 2 000 (červená čiara na obr. 1). V stĺpcoch *ProductRelated_Duration* a *PageValues* hodnoty rastú plynulo, tam som nemazal nič. Dve najväčšie hodnoty *ProductRelated_Duration* (43 171 a 63 974 sekúnd) sú vysoké, ale na krivku nadväzujú, tak som ich ponechal.

Tri hodnoty *SessionID* boli v dátach dvakrát s rovnakým riadkom, duplikát som zmazal. Počty odstránených riadkov sú v tab. 1.

Tab. 1: Odstránené outliery

Pravidlo	**Počet riadkov**
Administrative_Duration < 0	5
ProductRelated_Duration < 0	5
BounceRates mimo rozsahu 0 až 1	8
VisitorType = Admin	1
Administrative_Duration > 10 000	15
ProductRelated > 2 000	10
opakované *SessionID*	3
spolu	47

Po odstránení outlierov zostalo 12 286 riadkov.

1.3 Nepoužiteľné stĺpce a chýbajúce hodnoty

SessionID je len identifikátor návštevy a model sa z neho nemá čo naučiť, stĺpec som odstránil. Chýbajúce hodnoty boli v troch stĺpcoch (tab. 2).

Tab. 2: Chýbajúce hodnoty po odstránení outlierov

Stĺpec	**Chýba**	**Podiel**
Informational_Duration	8 105	65,97 %
ProductRelated_Duration	244	1,99 %
VisitorType	491	4,00 %

V stĺpci *Informational_Duration* chýbajú skoro dve tretiny hodnôt. Skúšal som, či sa dajú doplniť: ak návštevník neotvoril žiadnu informačnú stránku (*Informational* = 0), čas na nich je nula. Takto sa dá doplniť 6 529 hodnôt, pri zvyšných 1 576 čas nepoznáme. Stĺpec som nakoniec odstránil celý. Počet informačných stránok zostáva v stĺpci *Informational*. Ostatné možnosti porovnávam v časti 1.10.

V stĺpcoch *ProductRelated_Duration* a *VisitorType* chýba len malá časť hodnôt, preto som zmazal riadky. Bolo ich 729 (v šiestich chýbali obe hodnoty naraz), čo je 5,9 % dát. Doplnenie najčastejšou hodnotou by pri *VisitorType* zmenilo pomer typov návštevníkov a pri takej malej

strate sa neoplatí. Po tomto kroku má dataset 11 557 riadkov a 17 stĺpcov, zostalo 93,7 % pôvodných dát.

Pri kontrole som si všimol, že hodnoty nechýbajú náhodne. Vo všetkých troch stĺpcoch chýbajú len v riadkoch, kde *Revenue* = False. Mazaním riadkov sa preto podiel nákupov zvýšil z 15,5 % na 16,5 %.

1.4 Kódovanie nečíselných stĺpcov

Stĺpce *Month* a *VisitorType* obsahujú text. Stĺpce *OperatingSystems*, *Browser*, *Region* a *TrafficType* sú zapísané číslom, ale označujú kategórie a väčšie číslo neznamená viac. Všetkých šesť som zakódoval metódou one-hot (*pd.get_dummies*), kde každá hodnota dostane vlastný stĺpec s 0 alebo 1. Mesiace som neprevádzal na čísla 2 až 12. Model by ich potom bral ako veľkosť, teda december by bol „viac“ než november. Podiel nákupov ale s poradím mesiaca nerastie rovnomerne, v novembri je 26,9 % a v decembri 13,4 % (časť 2.3). *Weekend* a *Revenue* som previedol z True/False na 1/0. Po kódovaní má dataset 73 vstupných stĺpcov.

1.5 Vstupy, výstup a rozdelenie dát

Do X idú všetky stĺpce okrem *Revenue*, y je stĺpec *Revenue*. Dáta delím funkciou *train_test_split*, ktorú volám dvakrát: najprv oddelím trénovacie dáta, potom zvyšok rozdelím na validačné a testovacie. Používam parameter *stratify*, aby bol podiel nákupov vo všetkých troch množinách rovnaký. Ako základ som zvolil pomer 70 / 15 / 15, ďalšie pomery sú v časti 1.8.

1.6 Škálovanie

Na škálovanie používam *StandardScaler*, ktorý od každého stĺpca odčíta priemer a vydelí ho smerodajnou odchýlkou. Scaler nastavujem len na trénovacej množine a tie isté hodnoty potom použijem na validačnú a testovaciu. Keby som ho nastavil na všetkých dátach, do trénovania by sa dostala informácia z testovacej množiny.

1.7 Jednoduchý model

Ako jednoduchý model som použil logistickú regresiu zo Sklearn (*LogisticRegression*, max_iter = 1000, ostatné parametre predvolené). Pre porovnanie: model, ktorý by vždy odpovedal „bez nákupu“, by mal úspešnosť 83,49 %, lebo toľko je v dátach návštev bez nákupu.

Logistická regresia validačnú množinu nepotrebuje, lebo pri jej trénovaní netreba rozhodovať, kedy skončiť. Dáta aj tak delím na tri časti: zadanie to vyžaduje a validačnú množinu neskôr používa neurónová sieť (časť 3). Pri jednoduchom modeli zostáva validačná časť nevyužitá. Oba modely tak pri rovnakom pomere trénujú a testujú sa na rovnako veľkých množinách.

1.8 Experimenty s rozdelením dát

Vyskúšal som šesť pomerov. Výsledok závisí aj od toho, ktoré riadky sa náhodne dostanú do ktorej množiny, preto som každý pomer spustil päťkrát s iným seedom (0 až 4). V tab. 3 je priemer z piatich behov.

Tab. 3: Úspešnosť logistickej regresie pri rôznom rozdelení (trénovacia / validačná / testovacia)

Rozdelenie	Train riadkov	Test riadkov	Train (%)	Test (%)	Odchýlka test
90 / 5 / 5	10 401	578	87,98	88,13	0,42
80 / 10 / 10	9 245	1 156	88,01	88,13	0,74
70 / 15 / 15	8 089	1 734	88,02	87,55	0,47
60 / 20 / 20	6 934	2 312	88,10	87,71	0,36
40 / 30 / 30	4 622	3 468	88,19	87,77	0,42
10 / 45 / 45	1 155	5 201	88,71	87,11	0,26

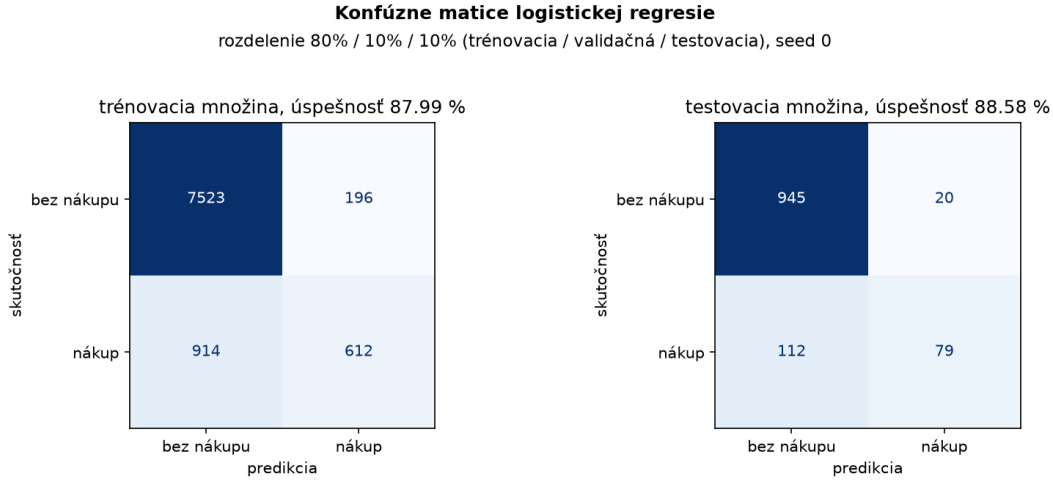
Pri pomeroch od 90 % do 40 % trénovacích dát sa úspešnosť na testovacej množine líši najviac o 0,6 bodu. Odchýlka medzi piatimi behmi toho istého pomeru je 0,3 až 0,7 bodu, takže tieto rozdiely pripisujem náhode. Logistická regresia má málo parametrov a 4 600 riadkov jej stačí podobne ako 10 400.

Rozdiel vidno až pri 10 % trénovacích dát. Úspešnosť na trénovacej množine stúpla na 88,71 %, na testovacej klesla na 87,11 % a rozdiel medzi nimi je 1,6 bodu. S 1 155 riadkami sa model viac prispôsobí trénovacím dátam a horšie zovšeobecňuje. Rozdelenia s malou testovacou množinou majú inú slabinu: pri 5 % má test len 578 riadkov a z toho 95 nákupov, takže každá chyba pohne výsledkom viac.

Všetky výsledky sú len o 3,6 až 4,6 bodu nad úrovňou 83,49 %. Prečo je to tak, vidno na konfúznej matici v časti 1.9.

1.9 Konfúzna matica najlepšieho modelu

Rozdelenia 90 / 5 / 5 a 80 / 10 / 10 majú rovnakú priemernú úspešnosť na testovacej množine (88,13 %). Vybral som 80 / 10 / 10, lebo má dvakrát väčšiu testovaciu množinu. Konfúzne matice na obr. 2 sú pre jeden beh tohto modelu (seed 0). Ten má na testovacej množine 88,58 %, o niečo viac než priemer piatich behov.



Obr. 2: Konfúzne matice logistickej regresie, rozdelenie 80 / 10 / 10

Na trénovacej množine model správne určil 7 523 návštev bez nákupu a 612 nákupov. Ďalších 914 nákupov označil ako návštevu bez nákupu, zachytil teda len 40 % nákupov. Na testovacej

množine je to 79 zo 191, čiže 41 %. Chyby opačným smerom sú zriedkavé (196 a 20). Model sa prikláňa k väčšinovej triede a úspešnosť okolo 88 % robia hlavne návštevy bez nákupu, ktorých je v dátach päťkrát viac.

1.10 Experiment so stĺpcom Informational_Duration

Vrátil som sa k stĺpcu *Informational_Duration* a porovnal tri možnosti: (A) stĺpec odstrániť, (B) doplniť nuly tam, kde *Informational* = 0, a zvyšné riadky zmazať, (C) doplniť nuly a zvyšok mediánom. Použil som rozdelenie 70 / 15 / 15 a znova päť behov (tab. 4).

Tab. 4: Porovnanie spôsobov, ako naložiť so stĺpcom Informational_Duration

Variant	**Riadkov**	**Nákupy (%)**	**Train (%)**	**Test (%)**
A: odstrániť stĺpec	11 557	16,51	88,02	87,55
B: nuly, zvyšok zmazať	10 086	18,92	88,44	88,67
C: nuly, zvyšok medián	11 557	16,51	88,32	87,96

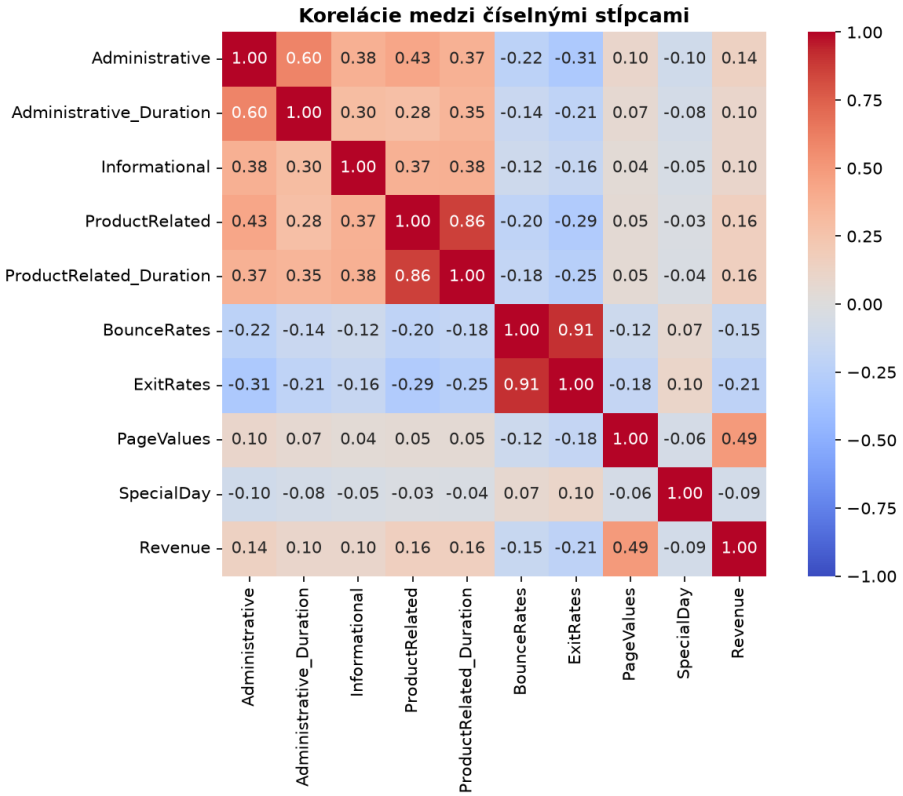
Variant B vyšiel najlepšie, na testovacej množine má 88,67 %. Tomuto výsledku ale neverím. Variant B maže 1 471 riadkov a všetky sú bez nákupu, lebo chýbajúce hodnoty sú len v takých riadkoch. Podiel nákupov tým stúpne na 18,9 %. Model tak dostal ľahšiu úlohu a testuje sa na iných dátach než vo variante A, čísla sa preto nedajú priamo porovnať.

Variant C je oproti A lepší o 0,4 bodu na testovacej množine, čo je v rámci rozptylu medzi behmi. Navyše 1 471 hodnôt doplnených mediánom patrí len návštevám bez nákupu a model by sa mohol naučiť spoznať práve túto jednu hodnotu. V ďalšej práci používam variant A.

2 Analýza dát (EDA)

Pri analýze pracujem s dátami po odstránení outlierov a chýbajúcich hodnôt, ale pred kódovaním, aby som mal k dispozícii slovné hodnoty (11 557 riadkov). V časti 2.6 používam pôvodný dataset.

2.1 Korelácie medzi číselnými stĺpcami



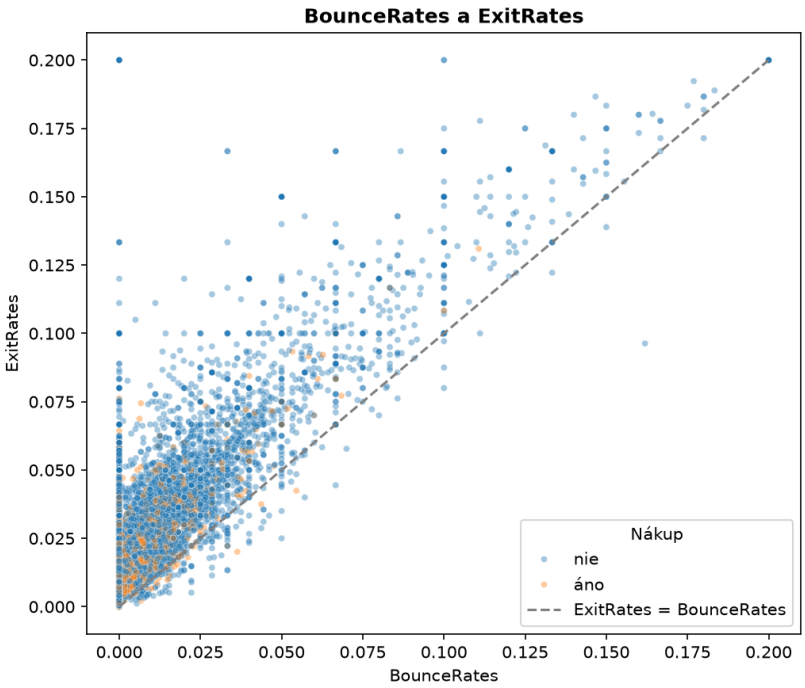
Obr. 3: Korelačná matica číselných stĺpcov a cieľovej premennej

Najsilnejšie spolu súvisia *BounceRates* a *ExitRates* (0,91) a potom *ProductRelated* a *ProductRelated_Duration* (0,86). Druhá dvojica je očakávaná: kto otvorí viac produktových stránok, strávi na nich viac času. Pri administratívnych stránkach je ten istý vzťah slabší (0,60). Pre model to znamená, že tieto dvojice nesú z veľkej časti rovnakú informáciu.

S *Revenue* koreluje výraznejšie len *PageValues* (0,49). Ostatné stĺpce majú koreláciu s nákupom medzi −0,21 a 0,16. Miery odchodu majú s počtom stránok zápornú koreláciu, návšteva s veľa stránkami má nižší *ExitRates*.

2.2 BounceRates a ExitRates

Dvojicu s najvyššou koreláciou som vykreslil bodovým grafom (obr. 4).

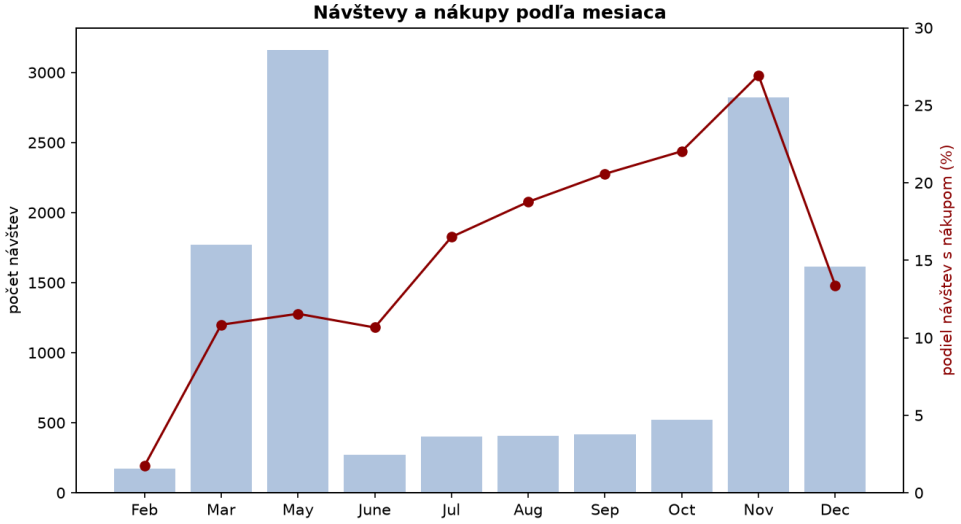


Obr. 4: BounceRates a ExitRates, farba podľa nákupu

Takmer všetky body ležia nad prerušovanou čiarou, *ExitRates* je teda skoro vždy väčší alebo rovný *BounceRates*. Pod čiarou je len 166 z 11 557 návštev. Dáva to zmysel, lebo okamžité opustenie stránky je zároveň aj odchod z nej. Zaujímavý je zvislý pás pri *BounceRates* = 0, kde *ExitRates* nadobúda hodnoty až po 0,2. Nulový bounce rate teda o miere odchodu veľa nepovie.

Nákupy (oranžové body) sú sústredené v ľavom dolnom rohu. Z 1 908 nákupov má len 17 hodnotu *BounceRates* nad 0,05.

2.3 Návštevnosť a nákupy podľa mesiaca



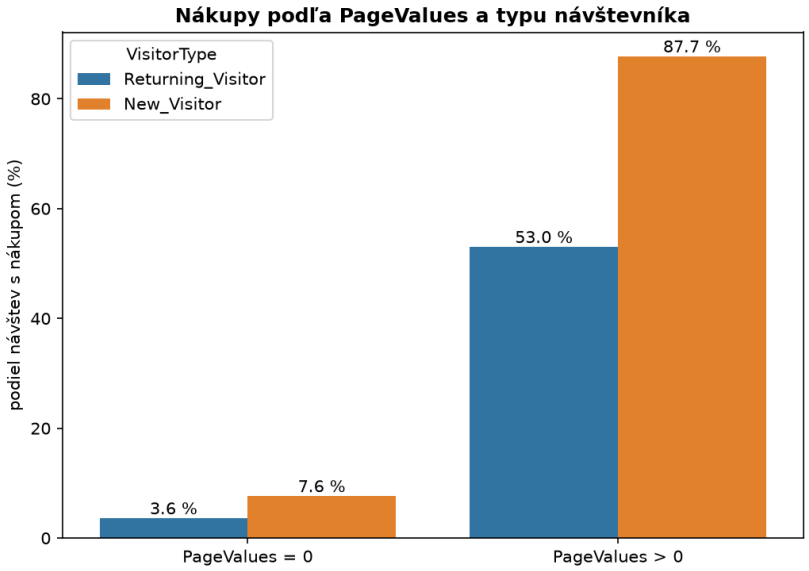
Obr. 5: Počet návštev (stĺpce) a podiel návštev s nákupom (čiara) podľa mesiaca

Najviac návštev bolo v máji (3 161), ale nákupom skončilo len 11,5 % z nich. November má podobný počet návštev (2 821) a podiel nákupov 26,9 %, najviac zo všetkých mesiacov. Od júla do októbra chodí na stránku málo ľudí, okolo 400 až 520 mesačne, podiel nákupov však každý mesiac rastie, zo 16,5 % na 22 %. V decembri klesne na 13,4 %. Február je s 1,7 % najslabší, má ale len 173 návštev.

Veľa návštev teda neznamená veľa nákupov. Stĺpec *SpecialDay* je nenulový len vo februári a v máji.

2.4 PageValues a typ návštevníka

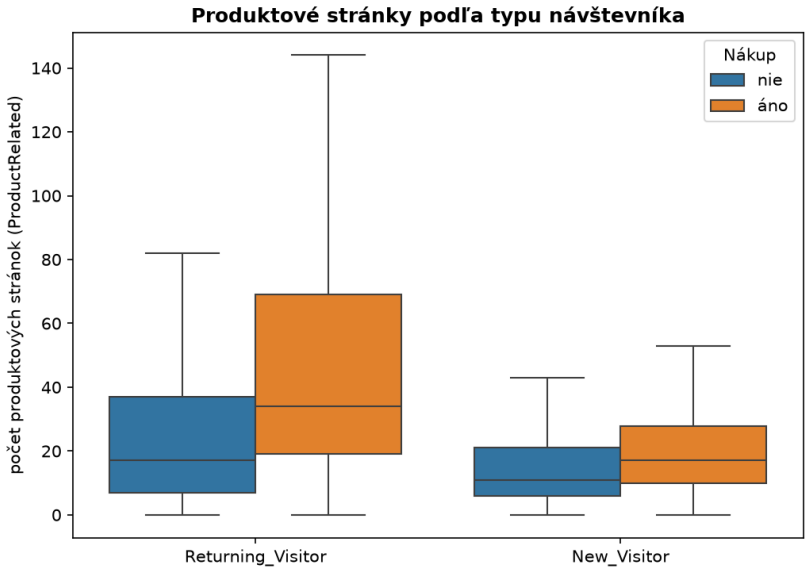
PageValues má s nákupom najsilnejší vzťah zo všetkých stĺpcov. Pozrel som sa, či je rovnaký pre nových a vracajúcich sa návštevníkov (obr. 6). Typ Other som vynechal, má len 78 riadkov.



Obr. 6: Podiel návštev s nákupom podľa PageValues a typu návštevníka

Ak je *PageValues* nulové, nákupom končí 3,6 % návštev vracajúcich sa a 7,6 % nových návštevníkov. Pri nenulovej hodnote je to 53,0 % a 87,7 %. Nový návštevník s nenulovým *PageValues* teda nakúpi skoro vždy, vracajúci sa len v polovici prípadov. Skupiny nie sú rovnako veľké: nových návštevníkov s nenulovým *PageValues* je 375, vracajúcich sa 2 254.

2.5 Počet produktových stránok podľa typu návštevníka

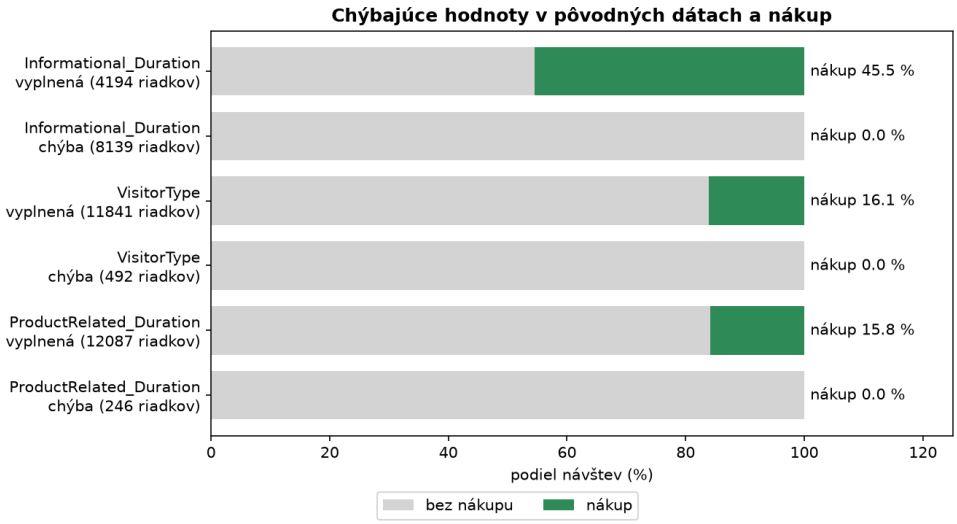


Obr. 7: Počet produktových stránok podľa typu návštevníka a nákupu, bez odľahlých bodov

Vracajúci sa návštevníci si prezrú viac stránok než noví. Medián je 17 oproti 11 pri návštevách bez nákupu a 34 oproti 17 pri návštevách s nákupom. V oboch skupinách majú návštevy s nákupom viac stránok. Nový návštevník pritom nakúpi po mediáne 17 stránok, čo je toľko, koľko si vracajúci sa návštevník prezrie aj bez nákupu. Noví návštevníci navyše nakupujú častejšie (26,5 % oproti 14,9 %), takže viac prezretých stránok samo o sebe nákup neznamená.

2.6 Chýbajúce hodnoty a nákup

Tento vzťah som našiel v pôvodnom datasete pri čistení (časť 1.3). V stĺpcoch *Informational_Duration*, *VisitorType* a *ProductRelated_Duration* chýba 8 139, 492 a 246 hodnôt a ani jeden z týchto riadkov nekončí nákupom. Pruhy pre chýbajúce hodnoty sú preto na obr. 8 celé sivé.



Obr. 8: Zloženie návštev podľa toho, či hodnota v stĺpci chýba (pôvodný dataset), každý pruh je 100 %

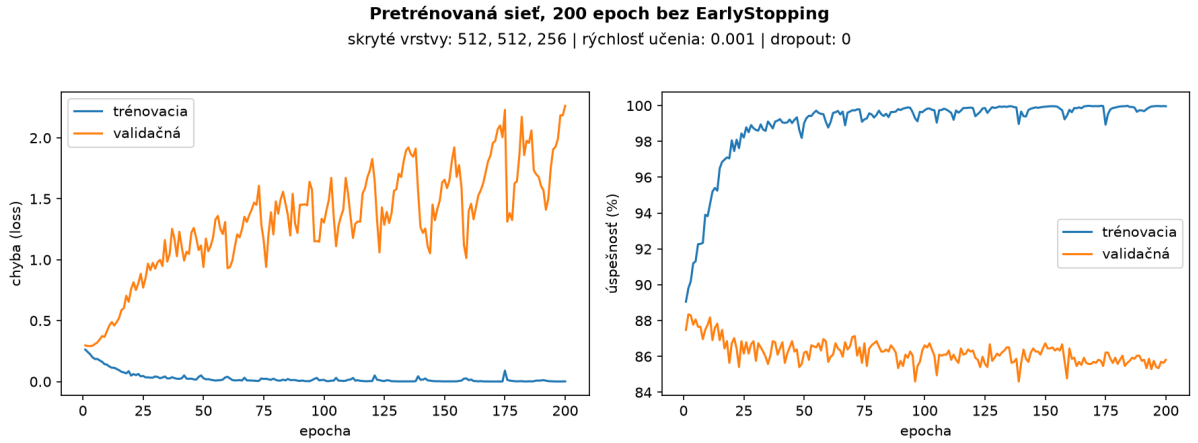
Najvýraznejšie je to pri *Informational_Duration*: tam, kde je hodnota vyplnená, končí nákupom 45,5 % návštev. V reálnych dátach by hodnoty takto chýbať nemali, vyzerá to na vlastnosť tohto datasetu. Informácia „hodnota chýba“ by modelu prezradila výsledok, preto som ju ako vstup nepoužil.

3 Neurónová sieť

Sieť je napísaná v PyTorch. Dáta sú pripravené rovnako ako v časti 1, s rozdelením 70 / 15 / 15 (8 089 trénovacích, 1 734 validačných a 1 734 testovacích riadkov) a 73 vstupmi. Používam plne prepojenú sieť s aktivačnou funkciou ReLU v skrytých vrstvách a jedným výstupným neurónom. Chybu počítam funkciou *BCEWithLogitsLoss* (binárna krížová entropia so sigmoidom), váhy upravuje optimalizátor Adam, veľkosť dávky je 64. Validačnú množinu používam po každej epoche na sledovanie priebehu trénovania, výsledky uvádzam pre trénovaciu a testovaciu množinu.

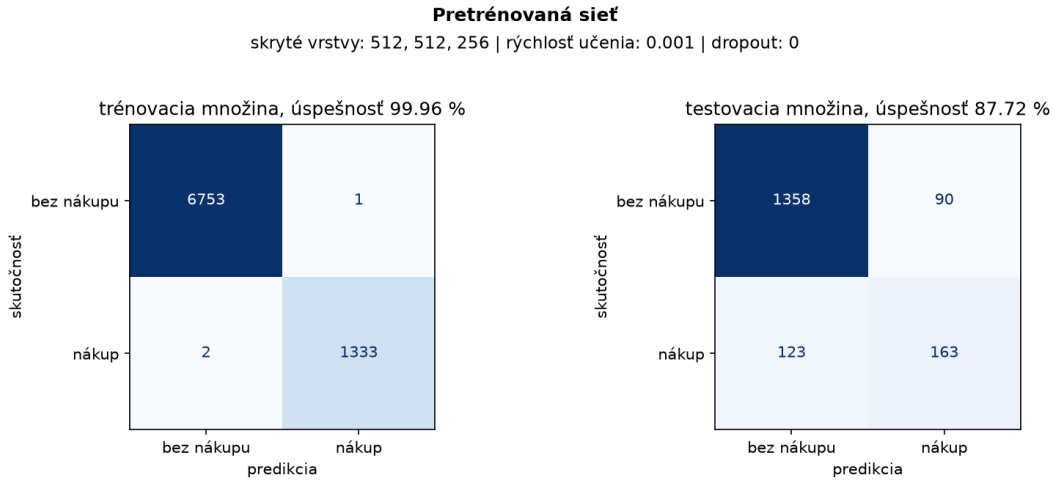
3.1 Pretrénovanie

Aby sa sieť pretrénovala, zvolil som zámerne veľkú architektúru: tri skryté vrstvy s 512, 512 a 256 neurónmi, spolu 432 129 parametrov na 8 089 trénovacích riadkov. Trénoval som 200 epoch s rýchlosťou učenia 0,001, bez dropoutu a bez zastavenia.



Obr. 9: Priebeh trénovania veľkej siete bez EarlyStopping

Chyba na trénovacej množine klesá skoro k nule (0,002 po 200 epochách). Validačná chyba rastie už od štvrtej epochy, z 0,29 na 2,26. Úspešnosť na trénovacej množine dosiahla 99,96 %, na validačnej klesla z 88,3 % na 85,8 %. Sieť si trénovacie dáta zapamätala.

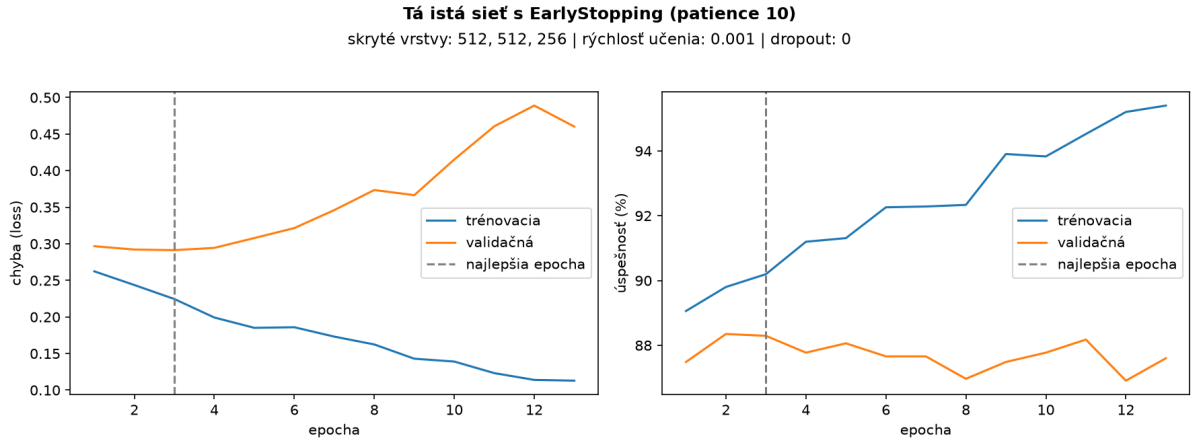


Obr. 10: Konfúzne matice pretrénovanej siete

Na trénovacej množine sieť urobila 3 chyby z 8 089. Na testovacej má úspešnosť 87,72 %, rozdiel oproti trénovacej je vyše 12 bodov. Z 286 nákupov v testovacej množine zachytila 163.

3.2 Odstránenie pretrénovania pomocou EarlyStopping

EarlyStopping som naprogramoval priamo v trénovacej slučke. Po každej epoche porovnám validačnú chybu s doteraz najlepšou. Ak je nižšia, uložím si váhy siete. Ak sa nezlepší 10 epoch po sebe (patience = 10), trénovanie zastavím a do siete vrátim váhy z najlepšej epochy. Architektúra a ostatné nastavenia sú rovnaké ako v časti 3.1.

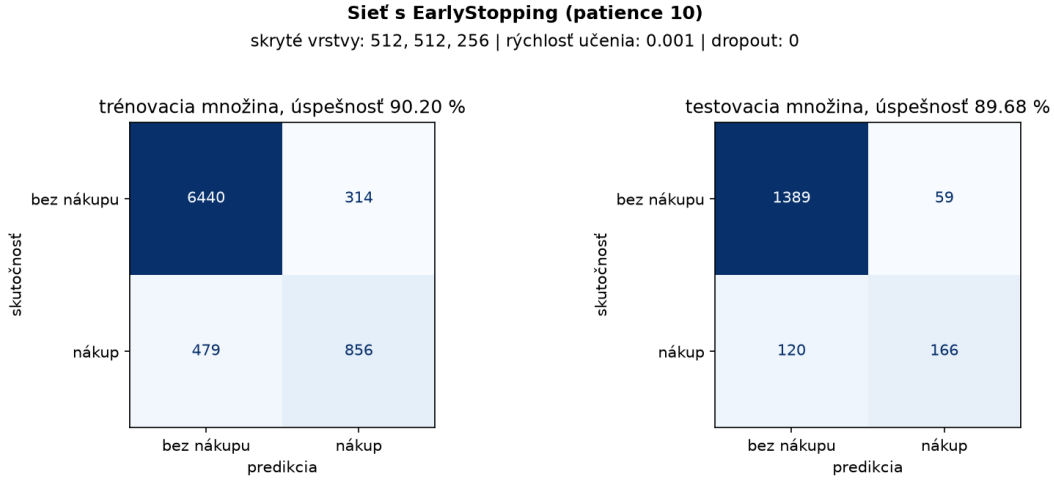


Obr. 11: Priebeh trénovania tej istej siete s EarlyStopping

Validačná chyba bola najnižšia v 3. epoche (0,291) a trénovanie sa zastavilo po 13. epoche. Na grafe vidno, že od 4. epochy sa krivky rozchádzajú rovnako ako predtým, sieť však skončí s váhami z 3. epochy.

Tab. 5: Veľká sieť bez EarlyStopping a s ním

Trénovanie	Odtrénovaných epoch	Váhy z epochy	Train (%)	Test (%)	Rozdiel
bez EarlyStopping	200	200	99,96	87,72	12,24
s EarlyStopping	13	3	90,20	89,68	0,52



Obr. 12: Konfúzne matice siete s EarlyStopping

Rozdiel medzi trénovacou a testovacou množinou klesol z 12,2 na 0,5 bodu a úspešnosť na testovacej množine stúpla o necelé 2 body. Na testovacej množine sieť zachytila 166 z 286 nákupov (58 %). Logistická regresia z časti 1 zachytila okolo 40 % nákupov.

3.3 Experimenty s hyperparametrami

Pre experimenty som zvolil menšiu sieť so skrytými vrstvami 64 a 32 neurónov a menil som rýchlosť učenia, architektúru a dropout. Všetky behy majú EarlyStopping s patience 10, najviac 200 epoch, dávku 64 a rovnaký seed. V tab. 6 sú nastavenia, ktoré sa menili, a dosiahnuté úspešnosti.

Tab. 6: Experimenty s neurónovou sieťou

Č.	Skryté vrstvy	Rýchlosť učenia	Dropout	Najlepšia epocha	Train (%)	Test (%)
1	64, 32	0,001	0	4	89,73	89,56
2	64, 32	0,01	0	2	89,31	89,56
3	64, 32	0,1	0	6	86,17	86,62
4	64, 32	0,0001	0	34	89,75	89,56
5	16	0,001	0	18	90,05	89,50
6	256, 128, 64	0,001	0	3	90,44	88,87
7	64, 32	0,001	0,3	6	89,68	90,02
8	256, 128, 64	0,001	0,3	5	90,26	89,85

Rýchlosť učenia (experimenty 1 až 4). Hodnoty 0,01, 0,001 a 0,0001 skončili na rovnakej úspešnosti 89,56 % a líšia sa hlavne počtom epoch. Pri 0,0001 trvalo 34 epoch, kým sieť

dosiahla to, čo pri 0,01 za dve. Pri 0,1 sú kroky príliš veľké a sieť sa učí zle, úspešnosť je 86,62 %.

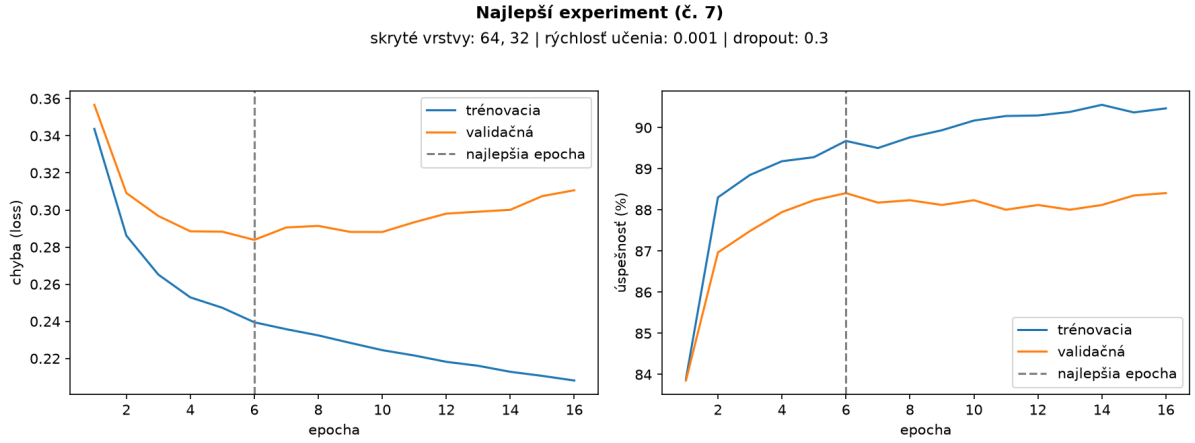
Architektúra (experimenty 5 a 6). Najmenšia sieť so 16 neurónmi má 89,50 %, skoro rovnako ako základná. Najväčšia má vyššiu úspešnosť na trénovacej množine (90,44 %) a nižšiu na testovacej (88,87 %), čo je náznak pretrénovania aj napriek EarlyStopping. Viac neurónov tu nepomáha.

Dropout (experimenty 7 a 8). Dropout 0,3 pomohol obom sieťam. Pri veľkej sieti stúpla úspešnosť na testovacej množine z 88,87 % na 89,85 % a základná sieť s dropoutom má najlepší výsledok, 90,02 %.

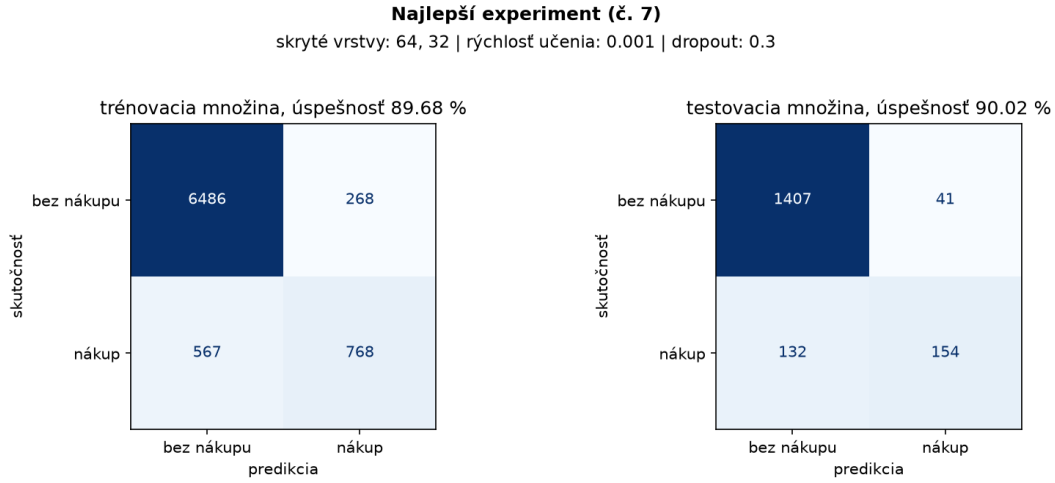
Rozdiely medzi väčšinou experimentov sú do pol bodu, čo je na testovacej množine s 1 734 riadkami 8 až 9 návštev. Každý experiment bežal len raz, takže poradie na prvých miestach by pri inom seede mohlo vyjsť inak. Jasne sa odlišuje len experiment 3.

3.4 Najlepší a najhorší experiment

Najlepší výsledok na testovacej množine má experiment 7 (vrstvy 64 a 32, rýchlosť učenia 0,001, dropout 0,3).



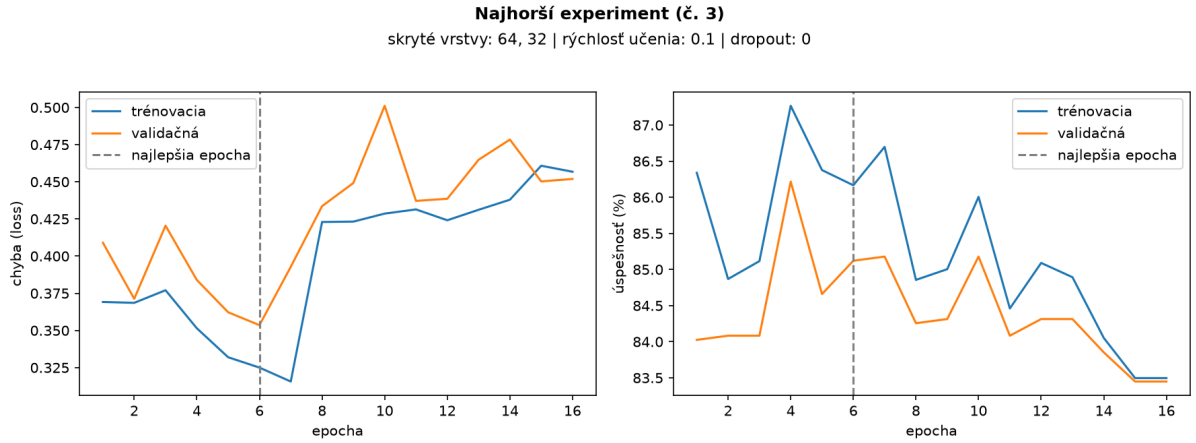
Obr. 13: Priebeh trénovania najlepšieho experimentu (č. 7)



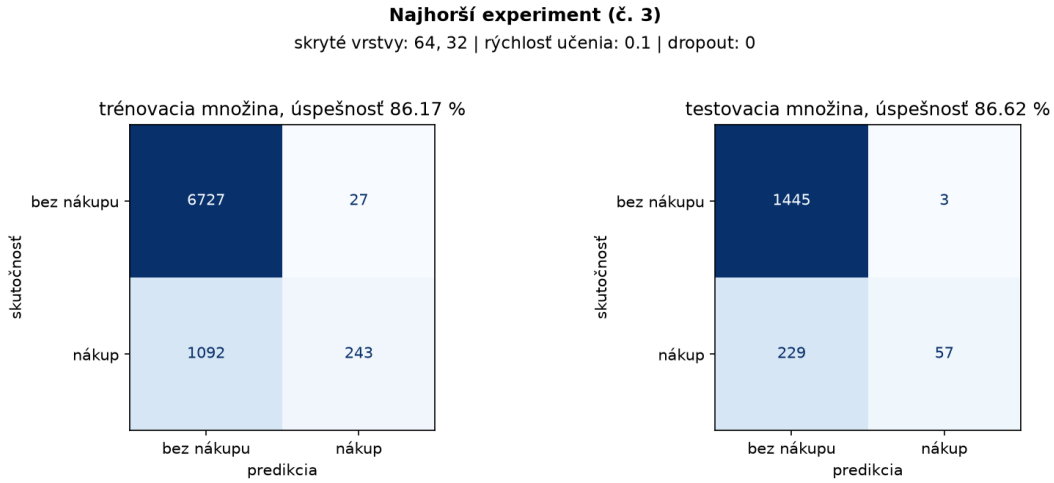
Obr. 14: Konfúzne matice najlepšieho experimentu (č. 7)

Validačná chyba bola najnižšia v 6. epoche a trénovanie sa zastavilo po 16. Trénovacia a validačná krivka idú blízko seba a rozchádzajú sa oveľa pomalšie než pri veľkej sieti z časti 3.1. Úspešnosť je 89,68 % na trénovacej a 90,02 % na testovacej množine. Na testovacej množine sieť zachytila 154 z 286 nákupov a 41 návštev bez nákupu označila ako nákup.

Najhorší je experiment 3 s rýchlosťou učenia 0,1.



Obr. 15: Priebeh trénovania najhoršieho experimentu (č. 3)



Obr. 16: Konfúzne matice najhoršieho experimentu (č. 3)

Chyba tu neklesá plynulo. Na trénovacej množine skáče medzi 0,32 a 0,46 a od 8. epochy je vyššia než po prvej epoche. Úspešnosť 86,62 % je len asi 3 body nad modelom, ktorý vždy odpovie „bez nákupu“. Z konfúznej matice vidno prečo: sieť označila ako nákup len 60 z 1 734 návštev a z 286 nákupov našla 57.

4 Nepovinné úlohy

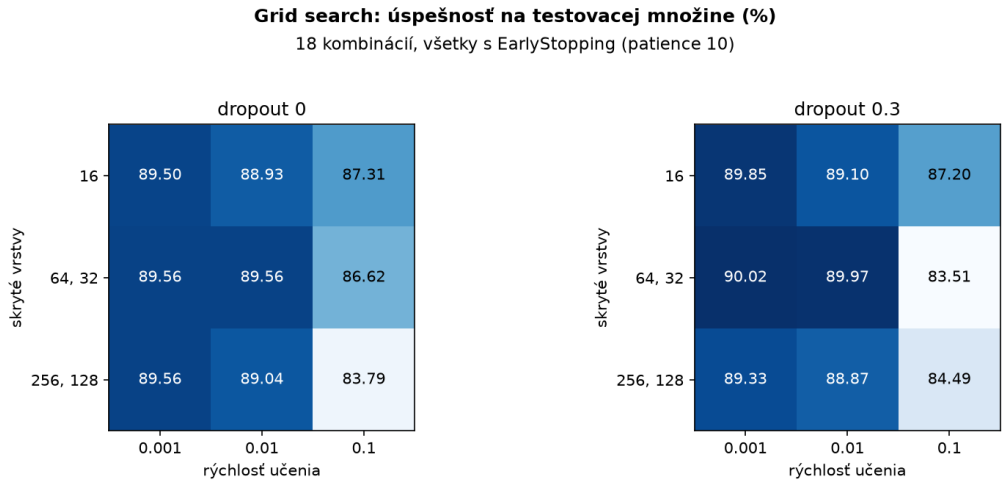
4.1 Grid search

Grid search skúša všetky kombinácie zvolených hodnôt hyperparametrov. Zvolil som tri architektúry (16 neurónov; 64 a 32; 256 a 128), tri rýchlosti učenia (0,1, 0,01 a 0,001) a dve

hodnoty dropoutu (0 a 0,3), spolu 18 kombinácií. Každá sieť sa trénuje funkciou z časti 3 s EarlyStopping (patience 10).

Tab. 7: Výsledky grid searchu, zoradené podľa úspešnosti na testovacej množine

Č.	**Skryté vrstvy**	**Rýchlosť učenia**	**Dropout**	**Najlepšia epocha**	**Train (%)**	**Test (%)**	**Train − test**
1	64, 32	0,001	0,3	6	89,68	90,02	−0,35
2	64, 32	0,01	0,3	8	89,75	89,97	−0,21
3	16	0,001	0,3	24	89,86	89,85	0,01
4	64, 32	0,01	0	2	89,31	89,56	−0,26
5	64, 32	0,001	0	4	89,73	89,56	0,17
6	256, 128	0,001	0	2	89,83	89,56	0,26
7	16	0,001	0	18	90,05	89,50	0,54
8	256, 128	0,001	0,3	2	89,62	89,33	0,28
9	16	0,01	0,3	3	89,08	89,10	−0,02
10	256, 128	0,01	0	3	89,80	89,04	0,76
11	16	0,01	0	5	90,11	88,93	1,18
12	256, 128	0,01	0,3	2	89,33	88,87	0,46
13	16	0,1	0	3	87,02	87,31	−0,29
14	16	0,1	0,3	1	86,98	87,20	−0,21
15	64, 32	0,1	0	6	86,17	86,62	−0,45
16	256, 128	0,1	0,3	2	84,30	84,49	−0,19
17	256, 128	0,1	0	6	83,89	83,79	0,10
18	64, 32	0,1	0,3	6	83,50	83,51	−0,01



Obr. 17: Úspešnosť na testovacej množine pre všetky kombinácie grid searchu

Najlepší výsledok. Najvyššiu úspešnosť na testovacej množine má sieť s vrstvami 64 a 32, rýchlosťou učenia 0,001 a dropoutom 0,3, s 89,68 % na trénovacej a 90,02 % na testovacej množine. Je to rovnaké nastavenie ako experiment 7 z časti 3.3. Hneď za ňou je tá istá sieť s rýchlosťou učenia 0,01 (89,97 %), rozdiel medzi nimi je jedna návšteva.

Najhorší výsledok. Posledných šesť miest patrí rýchlosti učenia 0,1 pri každej architektúre. Úplne posledná je sieť 64, 32 s dropoutom 0,3, ktorá má na testovacej množine 83,51 %. Táto

sieť neoznačila ako nákup ani jednu z 1 734 návštev a jej úspešnosť je len podiel návštev bez nákupu (1 448 z 1 734).

Pretrénovanie. Vďaka EarlyStopping je rozdiel medzi trénovacou a testovacou množinou všade malý, najviac 1,18 bodu (16 neurónov, rýchlosť učenia 0,01, bez dropoutu). Väčšie siete zastavili skoro, v 2. alebo 3. epoche, validačná chyba im teda začala rásť takmer hneď. V pároch, ktoré sa líšia len dropoutom, má verzia s dropoutom väčšinou menší rozdiel medzi trénovacou a testovacou množinou.

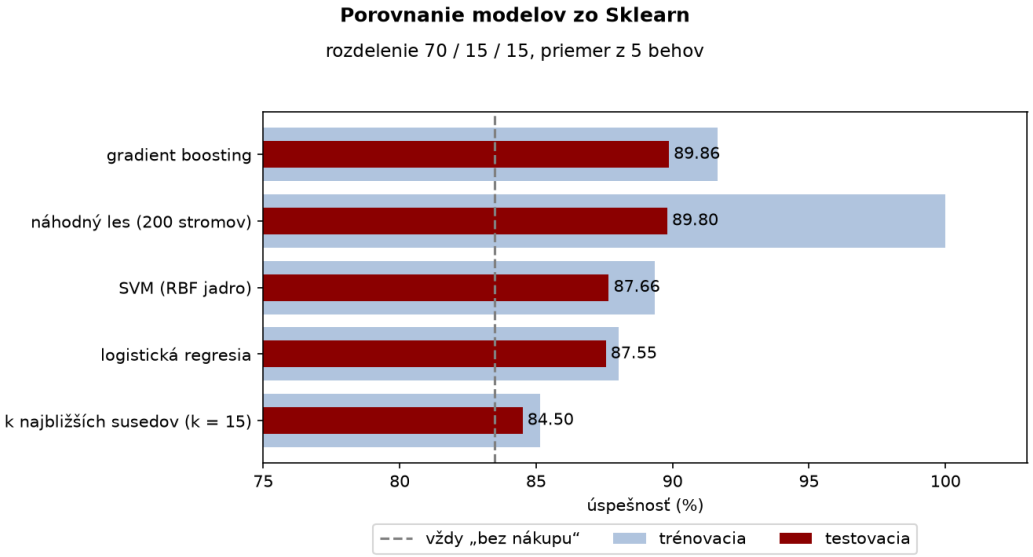
Z grid searchu vychádza to isté ako z experimentov v časti 3.3. Najviac záleží na rýchlosti učenia, najlepšie vychádza stredne veľká sieť s dropoutom a rozdiely medzi dobrými kombináciami sú okolo jedného bodu.

4.2 Porovnanie modelov zo Sklearn

Porovnal som päť modelov postavených na rôznych princípoch. Všetky dostali rovnaké dáta ako logistická regresia v časti 1: rozdelenie 70 / 15 / 15, škálovanie a päť behov s rôznym seedom. Modely majú predvolené nastavenia okrem parametrov uvedených v tab. 8.

Tab. 8: Porovnanie modelov, priemer z piatich behov, zoradené podľa úspešnosti na testovacej množine

Model	**Train (%)**	**Test (%)**	**Odchýlka test**	**Train − test**
gradient boosting	91,66	89,86	0,12	1,80
náhodný les (200 stromov)	100,00	89,80	0,33	10,20
SVM (RBF jadro)	89,35	87,66	0,55	1,69
logistická regresia	88,02	87,55	0,47	0,46
k najbližších susedov (k = 15)	85,15	84,50	0,44	0,65



Obr. 18: Úspešnosť modelov na trénovacej a testovacej množine

Logistická regresia je model z časti 1. Každému vstupu priradí jednu váhu a medzi triedami hľadá rovnú hranicu. Slúži tu ako základ na porovnanie.

Náhodný les je postavený z rozhodovacích stromov. Strom delí dáta otázkami typu „je *PageValues* väčšie než 1?“ a v každom liste predpovie prevažujúcu triedu. Les natrénuje 200 takých stromov, každý na náhodne vybraných riadkoch a stĺpcoch, a ich predpovede spojí hlasovaním. Jednotlivé stromy sú pretrénované, preto má les na trénovacej množine 100 %. Ich chyby sa ale navzájom rušia a na testovacej množine má 89,80 %.

Gradient boosting tiež používa stromy, ale malé a stavia ich postupne, každý ďalší opravuje chyby predošlých. Má najlepší výsledok na testovacej množine (89,86 %), najmenšiu odchýlku medzi behmi (0,12) a oproti náhodnému lesu malý rozdiel medzi trénovacou a testovacou množinou.

SVM s RBF jadrom hľadá hranicu medzi triedami, ktorá môže byť zakrivená. Výsledok 87,66 % je skoro rovnaký ako pri logistickej regresii (87,55 %). Trénovanie SVM trvalo zo všetkých modelov najdlhšie.

Metóda k najbližších susedov nájde 15 najpodobnejších trénovacích návštev a predpovie triedu, ktorá medzi nimi prevažuje. S 84,50 % je len bod nad modelom, ktorý vždy odpovie „bez nákupu“. Vzdialenosť sa počíta zo všetkých 73 stĺpcov rovnako, takže 63 stĺpcov z one-hot kódovania pravdepodobne prehluší tých pár, na ktorých záleží.

Najlepšie vychádzajú oba modely postavené na stromoch. Dosahujú okolo 90 %, podobne ako neurónová sieť z časti 3. Stromy navyše nepotrebujú škálovanie ani one-hot kódovanie, čo sa na tabuľkové dáta s jedným silným stĺpcom (*PageValues*) hodí.

Deklarácia použitia AI

Pri vypracovaní zadania som ako pomôcku používal jazykový model Claude od spoločnosti Anthropic (v nástroji Claude Code). Pomáhal mi s písaním kódu na predspracovanie dát, trénovanie modelov a vykresľovanie grafov, vysvetľoval mi použité metódy a výsledky a navrhoval text dokumentácie. O postupe som rozhodoval ja, napríklad ako naložiť s chýbajúcimi hodnotami, ktoré experimenty spustiť a ktoré modely porovnať. Kód som spúšťal, čísla v dokumentácii som kontroloval a celé riešenie som si prešiel tak, aby som mu rozumel.